



北京大学

本科生毕业论文

题目: **Householder 变换矩阵稳定性
定性的 Lean 形式化证明**

**Formalization of the Stability
Proof for the Householder
Transformation Matrix in Lean**

姓名: 蒋穆清

学号: 2200010766

院系: 数学科学学院

专业: 信息与计算科学

导师姓名: 夏壁灿

二〇二六年五月

版权声明

任何收存和保管本论文各种版本的单位和个人，未经本论文作者同意，不得将本论文转借他人，亦不得随意复制、抄录、拍照或以任何方式传播。否则，引起有碍作者著作权之问题，将可能承担法律责任。

摘要

随着形式化验证技术的发展，利用定理证明系统对数值算法进行严格验证，已成为提高计算结果可信性的重要研究方向。QR 分解是数值线性代数中的基础问题，在科学计算与工程应用中具有广泛用途。在求解 QR 分解问题的多种算法中，Householder 方法是能容忍浮点误差的、具有稳定性的重要方法。本文基于 Lean 定理证明系统，形式化证明了 Householder 变换在有误差的浮点运算系统中的稳定性。论文首先对浮点误差模型以及 Householder 变换中的关键计算过程进行了形式化描述，在此基础上完成了稳定性定理的证明，并给出了误差界的具体常数。研究表明，在合理的机器精度条件下，计算误差能够随向量维度保持线性增长，从而保证算法具有良好的数值稳定性。本文工作展示了 Lean 在数值线性代数算法验证中的应用能力，为后续数值算法的可信形式化研究提供了参考。

关键词：数学形式化；Lean；Householder 变换；QR 分解

ABSTRACT

With the development of formal verification, Lean has become an effective tool for rigorously analyzing and certifying numerical algorithms. QR decomposition is a fundamental problem in numerical linear algebra with wide applications in scientific computing and engineering. Among the various algorithms used to solve QR decomposition problems, the Householder method is renowned for its stability and tolerance to floating-point errors. This paper presents the formalization of the stability of Householder transformations using the Lean prover. This project formalized the floatpoint error framework and the essential computational steps involved in Householder transformations, and established a formal proof within Lean. Based on this formalization, explicit error bounds are derived, showing that under reasonable machine precision assumptions, the computational error grows linearly with the vector length. This result confirms the numerical stability of the algorithm and demonstrates the effectiveness of Lean in the formal verification of numerical linear algebra proofs, providing a foundation for further work on trustworthy and machine-checked numerical analysis.

KEYWORDS: Formalization, Lean, Householder transformation, QR decomposition

目 录

第一章 引言	1
1.1 研究背景	1
1.2 研究内容	1
第二章 理论基础	2
2.1 Householder 矩阵	2
2.1.1 定义与性质	2
2.1.2 Householder 变换	2
2.2 QR 分解	2
2.2.1 定义	3
2.2.2 Householder 矩阵在求解 QR 分解问题中的应用	3
2.2.3 实际计算中 Householder 法的细节	4
2.3 浮点计算	4
2.3.1 浮点数存储	5
2.3.2 舍入误差	5
2.3.3 浮点计算误差模型	5
第三章 Lean 基础	7
3.1 代码的基本结构	7
3.2 变量 (variable)	7
3.3 定义 (def)	8
3.3.1 定义函数	8
3.3.2 定义条件或关系	8
3.4 结构体 (structure)	9
3.5 函数	9
3.6 类型转换	9
3.7 小结	10
第四章 Householder 法的误差估计	11
4.1 记号	11
4.2 基础浮点运算的误差估计	12
4.3 Householder 过程	19
4.4 Householder 法的误差估计	22

第五章 总结	25
5.1 工作总结	25
5.2 未来展望	25
参考文献	26
附录	27
致谢	28

第一章 引言

1.1 研究背景

数值线性代数是科学计算的重要基础，在机器学习、计算机图形学、信号处理、工程仿真以及高性能计算等领域具有广泛应用。其中，QR 分解作为数值线性代数中的核心工具，为线性方程组求解、最小二乘问题、特征值计算等任务提供了统一而高效的计算框架。

Householder 变换是实现 QR 分解的重要方法。与经典 Gram-Schmidt 正交化相比，Householder 方法能够有效减小浮点舍入误差带来的影响，因此在实际计算中被广泛采用。其核心思想是通过构造正交反射矩阵，将向量逐步变换为目标方向，从而实现矩阵的正交化过程。由于 Householder 方法涉及大量浮点运算，其数值稳定性分析一直是数值分析领域的重要研究内容。

算法的数值稳定性证明往往包含复杂的符号推演与多层次误差传播分析。在验证这类包含大量近似和数值计算的证明时，人力验证容易出现疏漏。因此，人们需要使用能更严谨地验证推理过程正确性的形式化工具来完成证明。

近年来，定理证明辅助系统的发展为数学证明的形式化提供了有效工具。其中，Lean 作为一种基于依赖类型理论的交互式定理证明系统，兼具较强的表达能力与较高的自动化水平，已被广泛应用于数学定理验证、程序正确性证明以及形式化数学库构建等领域。通过 Lean 对数值算法的稳定性结论进行形式化，不仅能够提高证明过程的可靠性，也有助于推动可信科学计算的发展。

1.2 研究内容

本文围绕 Householder 变换的数值稳定性问题展开研究，并基于 Lean 定理证明系统对相关结论进行形式化验证。研究的核心问题是：在浮点误差模型下，Householder 变换中关键向量的计算结果是否能够保持足够小的相对误差，以及如何以形式化方式严格证明该性质。

为此，本文首先对 Householder 变换的数学定义及其稳定性理论进行了分析，并建立相应的浮点误差模型。在此基础上，使用 Lean 对相关数学对象、误差界以及稳定性定理进行形式化表示与证明。本文进一步给出了误差界中的具体常数，从而使稳定性结论具有更明确的量化形式。这表明形式化方法不仅能够验证已有理论结果，还能够补全传统分析中省略的隐式前提与证明步骤。

第二章 理论基础

本章及后文将统一采用 *Accuracy and Stability of Numerical Algorithms* [1] 中的记号。

2.1 Householder 矩阵

Householder 矩阵是数值线性代数中一种重要的正交变换工具，广泛应用于 QR 分解等矩阵计算算法中。其核心作用是通过一次“反射”变换，将向量或矩阵逐步转换为更便于计算的形式。相比 Gram-Schmidt 正交化等其他正交化方法，Householder 法具有更好的数值稳定性，因此在现代科学计算软件与高性能数值计算中被广泛采用。

2.1.1 定义与性质

Householder 矩阵（或称 Householder 变换矩阵）是具有如下形式的矩阵：

$$P = I - \frac{2}{v^T v} v v^T, \quad 0 \neq v \in \mathbb{R}^n$$

显然 P 为对称矩阵。另一方面，由

$$\begin{aligned} P^2 &= I - \frac{4}{v^T v} v v^T + \frac{4}{(v^T v)^2} v v^T v v^T \\ &= I - \frac{4}{v^T v} v v^T + \frac{4}{(v^T v)^2} v (v^T v) v^T = I \end{aligned}$$

结合其对称性易知 P 为正交矩阵。

2.1.2 Householder 变换

考虑方程 $Px = y$ 。其中 x, y 为常数，满足 $\|x\|_2 = \|y\|_2$ ，我们要求解满足方程的矩阵 P 。在这里我们令 P 为 Householder 矩阵，该方程可化为：

$$x - 2\left(\frac{v^T x}{v^T v}\right)v = y$$

令 $v = x - y$ ，易得

$$v^T v = x^T x + y^T y - 2x^T y$$

由 $\|x\|_2 = \|y\|_2$ ，即 $x^T x = y^T y$ ，故

$$v^T x = x^T x - x^T y = \frac{1}{2} v^T v$$

从而 $v = x - y$ 及其对应的 $P = I - \frac{2}{v^T v} v v^T$ 构成了方程的一组解。需要注意的是，当 $x = y$ 时，上述公式计算会导致分母为零，此时将 P 退化定义为单位矩阵 I 。

2.2 QR 分解

QR 分解是数值线性代数中的一种重要矩阵分解方法，在科学计算、数据分析、机器学习以及工程计算等领域具有广泛应用。通过 QR 分解，可以将一个复杂矩阵转化为更易处理的

形式，从而便于求解线性方程组、最小二乘问题以及特征值问题等计算任务。QR 分解在实际任务中非常重要，是现代数值算法中的基础工具之一。

2.2.1 定义

对于任意矩阵 $A \in \mathbb{R}^{m \times n}$ 且满足 $m \geq n$ ，总存在如下形式的 QR 分解：

$$A = QR = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} R_1 \\ 0 \end{bmatrix} = Q_1 R_1$$

其中 $Q \in \mathbb{R}^{m \times m}$ 为正交矩阵， $R_1 \in \mathbb{R}^{n \times n}$ 为上三角矩阵。

2.2.2 Householder 矩阵在求解 QR 分解问题中的应用

下面以一个 4×3 矩阵为例，演示利用 Householder 变换实现 QR 分解的消元过程。

$$A = \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{P_1} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{P_2} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & \times \end{bmatrix} \xrightarrow{P_3} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & 0 \end{bmatrix} = R$$

一般地，考虑如下的迭代消元过程：记 $A_1 = A$ ，在第 k 次迭代中矩阵具有以下分块形式：

$$A_k = \begin{bmatrix} R_{k-1} & z_k & B_k \\ 0 & x_k & C_k \end{bmatrix}, \quad R_{k-1} \in \mathbb{R}^{(k-1) \times (k-1)}, x_k \in \mathbb{R}^{n-k+1}$$

其中 R_{k-1} 为上三角矩阵。构造正交矩阵：

$$P_k = \begin{bmatrix} I_{k-1} & 0 \\ 0 & \widetilde{P}_k \end{bmatrix}$$

其中 $\widetilde{P}_k$ 为使 $\widetilde{P}_k x_k$ 除第一分量外均为 0 的正交矩阵。我们可以构造一个和 x_k 等长的、只有第一分量不为 0 的向量 $\sigma_k e_1$ ，并用 Householder 法求解方程 $\widetilde{P}_k x_k = \sigma_k e_1$ 来得到满足要求的 $\widetilde{P}_k$ 。这里 e_1 为第一分量上的单位向量 $(1, 0, \dots, 0)$ ， $\sigma_k = \pm \|x_k\|_2$ 以保证 $\|\sigma_k e_1\|_2 = \|x_k\|_2$ 。我们将在下一小节中讨论 σ_k 的符号选择。

我们将 P_k 作用于 A_k ，得到下一次迭代的矩阵 A_{k+1} ：

$$A_{k+1} = P_k A_k = \begin{bmatrix} I_{k-1} & 0 \\ 0 & \widetilde{P}_k \end{bmatrix} \begin{bmatrix} R_{k-1} & z_k & B_k \\ 0 & x_k & C_k \end{bmatrix} = \begin{bmatrix} R_{k-1} & z_k & B_k \\ 0 & \sigma_k e_1 & \widetilde{P}_k C_k \end{bmatrix}$$

$\sigma_k e_1$ 的后 $n-k$ 个分量均为 0，因此 A_{k+1} 满足下一轮的迭代要求。迭代过程全部结束后，我们取 $R = R_n$ 为上三角矩阵， $Q = P_1 P_2 \cdots P_n$ 为一系列正交矩阵之积，因此也是正交矩阵。

我们就完成了对 A 的 QR 分解。

2.2.3 实际计算中 Householder 法的细节

在上一节的迭代过程描述中, σ 的符号选取尚未明确指定。理论上, 对于给定的 x , $\sigma = \pm \|x\|_2$ 的两种符号均可满足 $\|\sigma e_1\|_2 = \|x\|_2$ 的要求, 从而都能构造出合法的 Householder 变换。但若 x 仅第一个分量不为零, 即 $x = (c, 0, \dots, 0)$, 且 σ 取与 c 同号的符号, 则 x_1 与 σ 的符号相同, 此时由 Householder 算法构造的向量 v 将退化为零向量, 进而导致计算归一化因子 $\frac{2}{v^T v}$ 时分母为零。为避免这一退化情形, 本项目中采用如下约定:

$$\sigma = -\text{sign}(x)\|x\|_2$$

其中 $\text{sign}(x)$ 的定义如 4.4 所示。当 $x_1 < 0$ 时取 -1 , $x_1 \geq 0$ 时取 1 。在这一约定下, 只要 $x \neq 0$, 或者 x_1 与 σ 均非零且异号, 或者 x_1 为零从而 $x \neq (c, 0, \dots, 0)$ 。因此 v 一定非零。

需要指出的是, 也存在其他合理的符号选取方式, 只是在本项目中我们采用了如上的方式。

另外, 在实际计算中, 显式构造并存储完整的因式矩阵 P_k 既耗时也无必要。注意到在 QR 分解的迭代过程中, P_k 始终以左乘的形式作用于某矩阵子块。由 Householder 矩阵的定义可知:

$$\widetilde{P}_k = I - \frac{2}{v^T v} v v^T,$$

因此对于任意矩阵 X ,

$$\widetilde{P}_k X = \left(I - \frac{2}{v^T v} v v^T \right) X = X - \left(\frac{2}{v^T v} v \right) (v^T X).$$

该表达式仅涉及向量 v 与矩阵 X 之间的数乘和乘法运算, 无需显式构建 $\widetilde{P}_k$ 。因此, 实际实现中只需存储每次迭代得到的向量 v_k , 即可通过上述公式高效地完成对后继矩阵的更新。因此我们在本项目中不关注 P 的精度, 而是直接分析向量 v 的计算误差。

2.3 浮点计算

实数的基本运算在各类科学计算中无疑扮演着至关重要的角色。然而, 受限于物理硬件的有限容量, 实数在机器内的表示不可避免地引入了截断与近似。工程实践通常采用浮点格式表示实数, 该表示方法采用科学计数法的思想, 利用尾数与指数的组合实现了对跨度宽阔数量级数值的统一表达, 奠定了当代大规模数值计算的工程基础。

尽管此种方案极大提升了计算机处理实数的计算能效, 针对浮点误差的研究长期以来仍是数值分析领域的前沿和重难点内容。一方面, 计算机由于仅能承载有限精度的数据, 故无论处于数据存取、制式转换抑或实数运算的何一环节都会有引入微小偏差的潜在可能。再者, 伴随大量循环与迭代结构, 上述误差不仅累加而且容易急剧传播放缩, 最终从根本上限制作答的可靠度乃至稳定性。高精度科研仿真抑或智能模型优化中更容不得轻视: 不经意引入的截尾畸变很容易酿成算法失败或是模型崩溃等恶劣后果。有鉴于此, 探明浮点误差其形成缘由和蔓延特点对于全维度保障演算的信度以及算法系统的鲁棒性无疑具有巨大的研究价值。

2.3.1 浮点数存储

计算机中的浮点数遵循 IEEE 754 标准，其存储结构主要由符号位、指数位和尾数位三部分构成。其中，符号位用于表示数值正负；指数位用于表示数值数量级；尾数位用于存储有效数字。以单精度浮点数为例，其使用 32 个比特位存储一个实数。大部分规格化浮点数被表示为： $(-1)^s \times (1.m) \times 2^{e-127}$

其中 s, e, m 均为相应的二进制分段。它们合计占据 32 位以构成完整的单精度浮点格式：段 s 专门承载符号信息，占 1 个比特位；段 e 作为阶码存放，占 8 个比特位；段 m 用作底数小数储存，占剩余的 23 个比特位。

2.3.2 舍入误差

即使参与运算的数据能够被浮点数精确表示，其代数运算的结果通常也无法以相同的有效位数精确存储。因此，必须将理论结果近似为当前存储格式下最接近的浮点数，这便导致了舍入误差的产生。

为说明舍入误差的产生机制，考虑一简化的 8 位浮点数模型（包含 1 位符号位、3 位指数位和 4 位尾数位）。对于如下加法运算：

$$2^4 \times 1.1110 + 2^3 \times 1.0011 = 30 + 9.5 = 39.5 = 2^5 \times 1.001111$$

上述理论结果的尾数需要 6 位，超出了模型中 4 位尾数的存储限制，故必须对其进行舍入截断：

$$2^5 \times 1.001111 \approx 2^5 \times 1.0100 = 40$$

受限于计算机的有限字长，产生舍入误差不可避免，但该误差通常有界且满足特定的数学规律。这为建立严谨的代数边界模型、定量分析算法的数值稳定性提供了理论依据。

2.3.3 浮点计算误差模型

舍入误差的边界受限于浮点格式的尾数长度。假设浮点系统提供 m 位尾数，则单次舍入操作产生的相对误差不超过 $1/2^{m+1}$ 。在此基础上，数值分析中常引入如下标准浮点误差模型以量化计算偏差：

设 $x, y \in \mathbb{R}$ ，记 $op \in \{+, -, \times, /\}$ 为基本浮点运算符号。一次标准浮点运算的计算结果可表示为：

$$fl(x \text{ op } y) = (x \text{ op } y)(1 + \delta), \quad |\delta| \leq u$$

其中 $fl(\cdot)$ 表示机器系统内的浮点算术实现， δ 为该次运算产生的相对舍入误差， u 称为机器精度，是一个完全由硬件存储精度决定的正常数。

需要注意的是，某些特定的基础操作（如取相反数、与零相加）不涉及尾数的舍入截断，从而不会引入舍入误差。

该模型的优势在于，它能够将一次浮点运算产生的误差统一表示为一个有界的相对扰动，从而使复杂算法中的误差传播分析可以转化为对多个扰动项的组合估计。相比直接分析底层二进制表示，该模型在保留主要误差特征的同时，大幅降低了理论分析的复杂度。此外，该模型中的误差界仅依赖于机器精度，而与具体运算数值无关，因此分析得到的结果对输入是

鲁棒的。在后续证明中，本文将反复利用该模型对连续浮点运算中的误差进行估计，并给出误差的上界。

第三章 Lean 基础

本章将简要介绍阅读形式化代码所需的 Lean 4 基础知识。我们并不深入讨论 Lean 的类型系统和证明策略，而是聚焦于理解项目中代码的语法结构，包括变量与参数的定义、定理与引理的声明方式、函数的定义方法以及复合数据结构的封装形式。读者仅需具备最基本的编程概念（如函数、参数、类型）即可顺利阅读。

3.1 代码的基本结构

Lean 中每一行代码都具有明确的类型。我们可以将每一个代码片段视作一个“声明”，它要么定义一个值或函数，要么声明一个待证明的性质。

考虑如下最常见的模式——引理（lemma）或定理（theorem）的声明：

```
lemma add_zero (a : ℕ) : a + 0 = a
```

引理：给定一个自然数 a ，可以推出 $a + 0 = a$ 。

我们来分析其结构：

- **lemma** 或 **theorem**：关键字，声明一条需要证明的命题。两者功能相同，**lemma**通常用于辅助性的小结论，**theorem**用于核心结论。
- **add_zero**：该引理的名称。
- 括号内的部分如 **(a : ℕ)** 称为显式参数。每个参数包含一个名字（如 a ）和它的类型（如 $\mathbb{N}$ ，即自然数集）。在调用该引理时，必须提供参数的具体值。
- 冒号后的部分 $a + 0 = a$ 称为该引理的结论，它本身是一个命题（Lean 中记为 **Prop**），即 $a + 0 = a$ 这个陈述。

整体来看，**add_zero** 是一个以 a 为输入、返回一个命题（此处为 $a + 0 = a$ ）的“函数”。如果提供了一个自然数类型的变量 a ，我们就得到了这个命题的一个证明。

有时参数会被大括号而非小括号扩起，如：

```
def identity {α : Type} (x : α) : α := x
```

这里的 **{α : Type}** 是隐式参数。调用时通常不需要提供 α ，Lean 会自动从后续参数推断出 α （这里可以从 x 的类型推断）。这个定义声明了一个恒等函数：对于任意类型 α ，给定一个 x 类型为 α ，它原样返回 x 。

3.2 变量（variable）

在 Lean 中，**variable** 用于声明一个在当前代码段中全局可用的变量，而不需要每次都在函数或定理的参数列表中重复写出。

```
variable (u : NNReal)
```

这声明了一个名为 u 的变量，它的类型是 **NNReal**（即非负实数）。在这之后所有位于同一代码段内的定义、引理和定理都可以直接使用 u ，而不必在每次声明时都将其作为参数列出。例如后面将会看到的 γ 的定义和 **gamma_nonneg** 引理中， u 都作为自由变量直接出现。在本项目中， u 代表机器精度，所有误差分析都以其为基础。

3.3 定义 (def)

def关键字用于定义一个新的常量或函数。根据定义的结果类型，它既可以定义一个计算用的函数，也可以定义一个描述性质的命题。

3.3.1 定义函数

以下是定义 4.3 的代码，定义了一个函数 γ ：

```
def  $\gamma$  (n :  $\mathbb{N}$ ) :  $\mathbb{R}$  := (n :  $\mathbb{R}$ ) * (u :  $\mathbb{R}$ ) / (1 - (n :  $\mathbb{R}$ ) * (u :  $\mathbb{R}$ ))
```

我们来逐段分析：

- **def**：关键字，表示定义一个值或函数。
- γ ：定义的函数的名称。
- (n : $\mathbb{N}$)：参数，n的类型是自然数 $\mathbb{N}$ 。
- : $\mathbb{R}$ ：冒号后的 $\mathbb{R}$ 表示函数返回值类型是实数。
- :=：赋值符号，表示“定义为”。
- 右侧为具体的表达式：(n : $\mathbb{R}$) * (u : $\mathbb{R}$) / (1 - (n : $\mathbb{R}$) * (u : $\mathbb{R}$))。注意这里用 (n : $\mathbb{R}$) 将自然数n转换为实数，这是必要的，因为乘法和除法运算要求操作数类型一致。

这个定义直接对应本文中的记号 $\gamma_n = \frac{nu}{1-nu}$ ，用于量化浮点运算的累积相对误差上界。

再看向量的定义：

```
abbrev Vec (n :  $\mathbb{N}$ ) := Fin n  $\rightarrow$   $\mathbb{R}$ 
```

abbrev是**def**的一种变体，它定义了一个类型别名。Fin n是包含n个元素的有限类型（索引从0开始），Fin n $\rightarrow$ $\mathbb{R}$ 表示从 $\{0, \dots, n-1\}$ 到实数的函数，即一个维度为n的实数向量。因此Vec n就是“长度为n的实数向量”这一类型的简写。

3.3.2 定义条件或关系

另一种常见的用法是用**def**定义一个命题（即**Prop**类型的值）。这类定义并不执行计算，而是描述一个条件或关系。我们考察定义 4.6 的代码：

```
def diff_gamma_bound (n :  $\mathbb{N}$ ) (x x' :  $\mathbb{R}$ ) : Prop :=  
  |x - x'|  $\leq$   $\gamma$  u n * |x|
```

这里的关键区别在于返回类型是**Prop**（命题）而非 $\mathbb{R}$ 。**Prop**是Lean中所有命题的类型。右侧的 $|x - x'| \leq \gamma u n * |x|$ 本身就是一个命题，它声明了 x' 与 x 的绝对误差不超过 γ_n 倍的 $|x|$ 。因此，diff_gamma_bound是一个谓词：当给定n x x'时，它表示“ x' 相对于 x 的误差满足n阶误差界”这一性质。在使用时，和使用函数的定义相似，只需将参数提供在命题名称后面，即可得到一个命题。例如diff_gamma_bound u (n+1) hv.1' hv_fl.1'就是一条具体的性质。

类似的，考虑定义 4.11：

```
def fl (x x' :  $\mathbb{R}$ ) : Prop :=  
   $\exists \delta : \mathbb{R}, |\delta| \leq (u : \mathbb{R}) \wedge x * (1 + \delta) = x'$ 
```

$\exists \delta : \mathbb{R}, \dots$ 是存在量词, 意为“存在一个实数 δ , 满足...”。 $\wedge$ 表示“且”。整个定义为: 实数 x' 是 x 的一次浮点运算结果, 即存在一个误差 δ 满足 $|\delta| \leq u$ 且 $x' = x * (1 + \delta)$ 。这正是上一章中描述的浮点误差模型。

3.4 结构体 (structure)

structure关键字用于将多个相关的数据打包成一个复合类型, 类似于其他语言中的类的概念。本项目中使用了一个结构体来封装 Householder 计算过程中的所有中间变量:

```
structure Householder (n : N) (hn : n ≠ 0) where
  l' : R
  l : R
  s : R
  v0 : R
  v' : Vec n
  p : R
  β : R
  b : R
  v : Vec n
```

这个结构体有两个参数: 向量长度 n 和一个证明 hn ($n \neq 0$, 用于确保向量非空)。它包含 9 个字段, 分别对应后文定义 4.20 中的 9 个变量。

这些字段按计算顺序依次排列, 每个字段的值依赖于前面的字段。结构体的作用是将所有这些变量打包为一个整体, 方便在后续的引理和定理中统一引用而不必逐个传递。例如, 后续定理中的参数 ($hv_f1 : Householder\ n\ hn$) 就代表一个包含了所有中间计算结果的“计算快照”。

3.5 函数

在 Lean 中, 一切都是函数。参数、定义、甚至定理本身都可以视为函数。以定义 4.4 为例:

```
def sign {n : N} (x : Vec n) (hn : n ≠ 0) : R :=
  if x < 0, Nat.pos_of_ne_zero hn < 0 then -1 else 1
```

这是一个典型的函数定义:

- 一个隐式参数 $\{n : N\}$ 表示向量长度。
- 两个显式参数: $(x : Vec\ n)$ (输入向量) 和 $(hn : n \neq 0)$ (即 n 非零的条件, 用于安全地访问向量的第一个元素)。
- 返回值类型是 $\mathbb{R}$ 。
- 函数体是 `if x < 0, Nat.pos_of_ne_zero hn < 0 then -1 else 1`: 如果向量的第一个分量 $x[0]$ 小于0则返回-1, 否则返回1。

3.6 类型转换

代码中频繁出现类似 $(n : \mathbb{R})$ 的写法, 这是 Lean 中的类型转换。我们重新来看 γ 的定义:

```
def γ (n : ℕ) : ℝ := (n : ℝ) * (u : ℝ) / (1 - (n : ℝ) * (u : ℝ))
```

此处 n 的类型是 $\mathbb{N}$ (自然数),但乘法运算要求两个操作数同为实数,因此需要用 $(n : \mathbb{R})$ 显式地将 n 从自然数类型转换为实数。

类似地,条件 $(hnu : ((n_1 + n_2) : \mathbb{R}) * (u : \mathbb{R}) < 1)$ 中, $((n_1 + n_2) : \mathbb{R})$ 也是将自然数的和转换为实数。

这种转换不会改变数值,仅改变类型信息。

3.7 小结

本章介绍了阅读本项目 Lean 代码所需的核心语法概念。总结如下:

- **variable:** 声明一个全局变量 (如机器精度 u), 在同一代码段中无需反复声明。
- **def:** 定义函数或性质。返回类型为`Prop`时是定义了一个描述性的命题 (谓词); 返回其他类型时是计算用的函数。
- **lemma**与**theorem:** 声明需要证明的命题。其参数分为显式参数 (圆括号) 和隐式参数 (花括号), 冒号后为结论。
- **structure:** 将多个相关的数据字段打包为一个复合类型, 便于整体传递。
- 类型转换 $(n : \mathbb{R})$: 将自然数 n 显式转换为实数, 以参与实数运算。

第四章 Householder 法的误差估计

本章阐述本研究的核心内容，即给出并证明利用 Householder 方法构造 Householder 矩阵时，核心向量 v 由于浮点舍入所引发的误差关于向量维度 n 与机器精度 u 的显式上界。

4.1 记号

定义 4.1. 向量

```
abbrev Vec (n : ℕ) := Fin n → ℝ
```

本文将向量定义为从有限指标集 $\{0, 1, \dots, n-1\}$ （在 Lean 中以 `Fin n` 类型表示）到实数域 $\mathbb{R}$ 的函数。换言之，一个长度为 n 的实数向量 x 可视为一个映射 $x : \{0, 1, \dots, n-1\} \rightarrow \mathbb{R}$ ，其中 $x(i)$ 表示向量的第 i 个分量。注意在 Lean 中 $x(i)$ 这种函数传参写作 `x i`。

在本项目中，出于以下两方面的考虑，我们采用这一函数式的向量定义，而不是使用 Lean 中 `EuclideanSpace` 类提供的欧几里得向量结构：

其一，在本研究的误差分析中，涉及向量的计算性质较多，较少借助 `EuclideanSpace` 类中的几何概念；

其二，该函数式定义与 Lean 中矩阵的定义方式直接相容。`Mathlib4` 中的 `Matrix` 类的定义即为 $M_{n \times m} : \{0, 1, \dots, n-1\} \times \{0, 1, \dots, m-1\} \rightarrow \mathbb{R}$ ，这使得向量与矩阵之间的运算能够自然地统一表达，从而简化本代码库在矩阵计算中以后可能的应用。

定义 4.2. 浮点计算系统的精度

```
variable (u : NNReal)
```

记 u 为机器精度，其类型为 `NNReal`（非负实数）。与第 2.3 节中定义相同， u 是一个完全由硬件浮点存储格式决定的正常数，代表在浮点运算中单次舍入操作可能引入的相对误差上限。注意 `variable (u : NNReal)` 的声明使得 u 作为全局变量可用，后文中的所有定义和引理均可直接引用该变量，无需在每次声明时重复传递。

定义 4.3. 浮点误差阶数

```
def γ (n : ℕ) : ℝ := (n : ℝ) * (u : ℝ) / (1 - (n : ℝ) * (u : ℝ))
```

定义函数

$$\gamma_n = \frac{nu}{1 - nu},$$

其中 n 为自然数， u 为机器精度。该函数在满足 $nu < 1$ 的条件下有意义，且分母为正。

我们之所以选取这一特定的表达式作为误差阶数的度量，而非采用更直观的简单上界 nu ，主要是基于以下两方面的考量。首先，该定义具有良好的可组合性：由下文引理 4.5 可知，对任意自然数 m, n ，有

$$(1 + \gamma_m)(1 + \gamma_n) < 1 + \gamma_{m+n},$$

即误差阶数在复合运算中满足可加性。若采用 nu 作为误差界，则 $(1 + mu)(1 + nu) = 1 + (m + n)u + mnu^2$ ，其超出 $1 + (m + n)u$ 的部分无法被该线性界所控制，因此不具备上述

可组合性质。

其次，易验证 $u < \gamma_1$ ，即 γ_1 相比单次机器精度是一个略为保守的上界，这保证了 γ 定义能够安全地替代 u 作为误差分析的基本度量。综上， γ_n 是进行误差阶数分析的一个十分便利的辅助工具，在后续推导中将反复使用。

在下文中，我们用 θ_n 表示一个不到 γ_n 的 n 阶的相对误差。即 θ_n 满足 $|\theta_n| \leq \gamma_n$ 。在后续的推导中，所有以 θ 标记的变量均默认满足这一条件，不再逐一说明。并且这些 θ 变量即使在同一处代码中下标相同且多次出现，它们也未必相等，而是多个独立的变量，每个变量都满足 $|\theta_n| \leq \gamma_n$ 。

定义 4.4. 符号函数

```
def sign {n : N} (x : Vec n) (hn : n ≠ 0) : R :=
  if x < 0, Nat.pos_of_ne_zero hn < 0 then -1 else 1
```

定义向量 x 的符号函数如下：

$$\text{sign}(x) = \begin{cases} 1, & x_1 \geq 0, \\ -1, & x_1 < 0, \end{cases}$$

其中 x_1 为向量 x 的第一个分量。该函数仅考虑向量的首分量，并以该分量的符号作为整个向量的符号标记。

需要特别说明的是，此处的 **sign** 函数在 $x_1 = 0$ 处的取值为 1，这与最标准的符号函数（通常在零点处取 0）不同。我们的符号函数选取源于 Householder 算法本身的计算需求，在构造 Householder 变换时，我们选取 $\sigma = -\text{sign}(x)\|x\|_2$ 作为目标值（详见第 2.2.3 小节）。

对于本研究中涉及的任意中间变量，本文采用如下约定加以区分：记 a 为精确计算所得的真值（即数学意义上的理论结果），而相应的 $\hat{a}$ 为在浮点环境中经过有限精度运算得到的近似计算值。在下文的误差分析中，我们始终以这一对记号展开讨论，并致力于刻画 Householder 法得到的向量 $\hat{v}$ 相对于真实向量 v 的误差之范围。

4.2 基础浮点运算的误差估计

本节将遵循源文件 `Basic.lean` 中的逻辑顺序，给出一系列关于浮点算术运算误差估计的基础结论，并证明后续推导所需的相关引理。

引理 4.5. 浮点误差阶数的可加性

```
lemma gamma_mul_le (n₁ n₂ : N)
  (hnu : ((n₁ + n₂) : R) * (u : R) < 1) :
  (γ u n₁ + 1) * (γ u n₂ + 1) ≤ γ u (n₁ + n₂) + 1
```

本引理给出了 γ 函数的可加性性质：对任意自然数 n_1, n_2 ，若 $(n_1 + n_2)u < 1$ （即分母为正），则成立

$$(1 + \gamma_{n_1})(1 + \gamma_{n_2}) \leq 1 + \gamma_{n_1+n_2}.$$

换言之， n_1 阶与 n_2 阶的相对误差相复合后，总误差阶数不超过 $n_1 + n_2$ 。这一性质是后

续所有复合运算误差分析的基础。它保证了我们可以将误差阶数像普通自然数一样相加，而无需担心复合后误差的非线性放大超出控制。

证明. 将 γ_n 的表达式代入，上述不等式等价于

$$\frac{1}{1 - un_1} \cdot \frac{1}{1 - un_2} \leq \frac{1}{1 - u(n_1 + n_2)}.$$

由前提条件 $(n_1 + n_2)u < 1$ 可知各分式的分母均为正数，故该不等式可通过简单的代数变形得到验证。 $\square$

定义 4.6. 误差界谓词

```
def diff_gamma_bound (n : N) (x x' : R) : Prop :=
  |x - x'| ≤ γ u n * |x|
```

为了在形式化系统中方便地描述“计算结果 x' 落在真实值 x 的 γ_n 相对误差界内”这一概念，我们定义了谓词 `diff_gamma_bound`。该谓词接受三个参数：误差阶数 n 、精确值 x 和计算值 x' ，其含义为

$$|x' - x| \leq \gamma_n |x|.$$

引理 4.7. 误差的保号性

```
lemma pos_diff_gamma_bound_pos (n : N) (x x' : R)
  (hnu : 2 * (n : R) * (u : R) < 1) (hx : x > 0)
  (hbound : diff_gamma_bound u n x x') :
  x' > 0
```

该引理断言：若精确值 x 为正实数，且其计算值 x' 满足 n 阶 γ 误差界，同时误差界参数满足 $2nu < 1$ ，则 x' 亦为正数。换言之，当机器精度足够高、运算次数足够少时，累积的舍入误差不足以改变计算结果的正负号。例如，在 **Householder** 算法的误差传递过程中，我们需要确保某些中间变量（如 s 与 x_1 ）保持同号，从而能够正确应用加法误差引理。

定理 4.8. 误差界的传递性

```
theorem diff_gamma_bound_trans (n1 n2 : N) (x x' x'' : R)
  (hnu : ((n1 + n2) : R) * (u : R) < 1)
  (hbound1 : diff_gamma_bound u n1 x x')
  (hbound2 : diff_gamma_bound u n2 x' x'') :
  diff_gamma_bound u (n1 + n2) x x''
```

本定理是引理 4.5 的直接应用。它描述了误差在复合运算中的传递规律：设真实值 x 经第一步运算得到近似值 x' ，满足 $|x' - x| \leq \gamma_{n_1} |x|$ ；该近似值 x' 再经第二步运算得到 x'' ，满足 $|x'' - x'| \leq \gamma_{n_2} |x'|$ 。此外，为保证 γ 函数的可组合性，还需假设 $((n_1 + n_2)u < 1)$ ，即分母为正且累加后的误差阶数仍有定义。在此条件下，最终结果 x'' 相对于原始真实值 x 的总误差满足

$$|x'' - x| \leq \gamma_{n_1+n_2} |x|.$$

在复合运算中，误差阶数满足可加性。即每次运算引入的误差阶数逐次叠加后，最终的总误差阶数不超过各单项运算误差阶数之和。这为我们后续对 Householder 算法中长达十余步的运算链进行逐层误差累加提供了理论依据。我们只需将每一步的误差阶数相加，即可得到最终结果的总误差阶数。

引理 4.9. 加法运算的误差界

```
lemma add_bound_nonneg
  (nx ny : ℕ) (x y x' y' : ℝ)
  (hx : x ≥ 0) (hy : y ≥ 0)
  (hnu : ((nx ⊔ ny) : ℝ) * (u : ℝ) < 1)
  (hboundx : diff_gamma_bound u nx x x')
  (hboundy : diff_gamma_bound u ny y y') :
  diff_gamma_bound u (nx ⊔ ny) (x + y) (x' + y')
```

现在考虑加法的误差传递。给定两个非负实数 x, y ，其浮点计算值 x', y' 分别满足 n_x 阶和 n_y 阶误差界。此外，还需假设 $(\max\{n_x, n_y\} \cdot u < 1)$ ，以确保 $\gamma_{\max\{n_x, n_y\}}$ 有定义。我们希望估计 $x' + y'$ 相对于 $x + y$ 的误差。

本引理断言：在此条件下， $x' + y'$ 的误差阶数由 n_x 与 n_y 中的较大者 $\max\{n_x, n_y\}$ 控制。这是因为加法的误差主要来源于两个加数各自的误差，而两者误差的绝对值之和自然受控于较大误差阶数与两者绝对值之和的乘积。

证明. 由三角不等式和误差界定义可得

$$\begin{aligned}
 |(x' + y') - (x + y)| &\leq |x' - x| + |y' - y| \\
 &\leq \gamma_{n_x}|x| + \gamma_{n_y}|y| \\
 &\leq \gamma_{\max\{n_x, n_y\}}(|x| + |y|) \\
 &= \gamma_{\max\{n_x, n_y\}}|x + y|,
 \end{aligned}$$

其中最后一个等式用到 $x, y \geq 0$ 的条件，从而 $|x| + |y| = x + y = |x + y|$ 。□

需要说明的是，上述推导中最后一个等式的成立依赖于 x, y 同号（此处均为非负）的条件。在 Lean 的形式化实现中，为满足该条件，加法误差界被拆分为非负和非正两个版本分别证明。

引理 4.10. 乘法运算的误差界

```
lemma mul_bound
  (nx ny : ℕ) (x y x' y' : ℝ)
  (hnu : ((nx + ny) : ℝ) * (u : ℝ) < 1)
  (hboundx : diff_gamma_bound u nx x x')
  (hboundy : diff_gamma_bound u ny y y') :
  diff_gamma_bound u (nx + ny) (x * y) (x' * y')
```

对于乘法运算，若 $x' = (1 + \theta_{n_x})x$ 且 $y' = (1 + \theta_{n_y})y$ ，且满足 $(n_x + n_y)u < 1$ （以保证

$\gamma_{n_x+n_y}$ 有定义), 则乘积 $x'y'$ 的误差界为 $n_x + n_y$ 阶。与加法不同, 乘法的误差阶数是两个因子误差阶数之和而非最大值——这是因为两个因子各自的相对误差在相乘时会相互耦合, 产生交叉项, 使得总的相对误差阶数上升为两者之和。

证明. 通过恒等变形和三角不等式进行估计:

$$\begin{aligned} |x'y' - xy| &\leq |x'y' - x'y| + |x'y - xy| \\ &\leq |x'||y' - y| + |y||x' - x| \\ &\leq (1 + \gamma_{n_x})|x| \cdot \gamma_{n_y}|y| + |y| \cdot \gamma_{n_x}|x| \\ &\leq (\gamma_{n_x} + \gamma_{n_y} + \gamma_{n_x}\gamma_{n_y})|x||y| \\ &\leq \gamma_{n_x+n_y}|xy|. \end{aligned}$$

其中最后一步使用了引理 4.5, 即 $1 + \gamma_{n_x+n_y} = (1 + \gamma_{n_x})(1 + \gamma_{n_y})$ 的等价变形。□

定义 4.11. 单次浮点运算的误差模型

```
def fl (x x' : ℝ) : Prop :=
  ∃ δ : ℝ, |δ| ≤ (u : ℝ) ∧ x * (1 + δ) = x'
```

与第 2.3 节中描述的浮点误差模型一致, 我们在此定义谓词 `fl` 来描述单次基本浮点运算产生的误差: 对于给定的精确运算结果 x 和浮点计算结果 x' , `fl u x x'` 成立当且仅当存在一个实数 δ 满足

$$|\delta| \leq u, \quad x' = x(1 + \delta).$$

该定义是后续所有误差分析的基础——无论是加法、乘法还是开方运算, 其浮点实现所引入的误差均以此为标准模型进行描述。

引理 4.12. 单次浮点误差与 γ 误差界的关系

```
lemma fl_to_gamma_bound (x x' : ℝ) (hu : (u : ℝ) < 1) :
  fl u x x' → diff_gamma_bound u 1 x x'
```

本引理将单次浮点误差模型与 γ 误差界联系起来。若 $u < 1$, 且浮点计算结果 x' 满足 `fl(x) = x'` (即 $|\delta| \leq u$, $x' = x(1 + \delta)$)。则单次浮点运算产生的误差可直接被 γ_1 误差界所控制, 即

$$|x' - x| \leq \gamma_1|x|.$$

证明此结论只需验证 $\gamma_1 > u$ 。由 γ_1 的定义 $\gamma_1 = u/(1 - u)$, 结合 $u < 1$ 即得。因此, $|\delta| \leq u < \gamma_1$, 从而 $|x' - x| \leq u|x| < \gamma_1|x|$ 。

这一引理的作用在于: 在后续复合运算的误差分析中, 我们可将每次基本浮点运算的误差统一用 γ 阶数来表示, 从而充分利用 γ 函数的可加性进行误差累积。

定义 4.13. 向量的 γ 误差界

```
def diff_gamma_bound_vec (n : ℕ) {m : ℕ} (x : Vec m) (x' : Vec m) :
  Prop := ∀ i, diff_gamma_bound u n (x i) (x' i)
```

前面定义的`diff_gamma_bound`仅适用于标量（实数）之间的误差关系。对于向量，我们将误差界概念逐分量推广：向量 x' 落在 x 的 n 阶误差界内，当且仅当对于每一个分量指标 i ，均有 x'_i 落在 x_i 的 n 阶误差界内，即

$$\forall i, |x'_i - x_i| \leq \gamma_n |x_i|.$$

该定义将作为后续判定 Householder 输出向量误差的最终标准。

定义 4.14. 基本浮点运算的谓词封装

```
def fl_add (x y z : ℝ) : Prop :=
  fl u (x + y) z
def fl_mul (x y z : ℝ) : Prop :=
  fl u (x * y) z
def fl_smul_vec (a : ℝ)
  (x : Vec n) (y : Vec n) : Prop :=
  ∀ i, fl u (a * x i) (y i)
```

以上三个定义是对基础浮点运算的谓词封装。`fl_add x y z` 表示 z 是 $x + y$ 的浮点计算结果，即将 z 表示成 $z = (x + y)(1 + \delta)$ 时，有 $|\delta| \leq u$ ；类似地，`fl_mul x y z` 表示 z 是 $x \times y$ 的浮点计算结果。`fl_smul_vec a x y` 表示向量 y 是标量 a 与向量 x 的数乘结果的浮点实现，即对每个分量 i 均有 y_i 是 $a \cdot x_i$ 的浮点计算结果。

在后续的 Householder 误差分析中，我们将通过这些封装好的谓词来描述每一步中间变量的浮点计算关系。

定义 4.15. 带浮点误差的向量点乘

```
def fl_vec_dot : {n : ℕ} → Vec n → Vec n → ℝ → Prop
| 0, _, _, z => z = 0
| n + 1, x, y, z =>
  ∃ p s : ℝ,
    fl_mul u (x 0) (y 0) p ∧
    fl_vec_dot
      (fun i => x (Fin.succ i))
      (fun i => y (Fin.succ i)) s ∧
    fl_add u p s z
```

为了描述向量点乘运算在浮点环境中的计算结果，我们递归地定义了谓词`fl_vec_dot`。其递归结构直接对应点乘的计算过程：

- 当向量长度 $n = 0$ 时，点乘结果为 0。
- 当向量长度 $n > 0$ 时，先计算第一个分量 x_1 与 y_1 的乘积 p （满足`fl_mul`），再递归计算剩余 $n - 1$ 个分量的点乘结果 s （满足`fl_vec_dot`），最后将 p 与 s 相加得到最终结果 z （满足`fl_add`）。

每一步基本运算（乘法和加法）均被约束在单次浮点误差界 u 以内。

需要说明的是，上述递归定义在 $n = 1$ 时会执行一次额外的加法 $p + 0$ ，且没有假定该加法运算的误差为零。这使得最终结果的理论误差上界由通常的 n 阶放宽至 $n + 1$ 阶。虽然这一放宽在理论上并不必要，但它简化了形式化证明的递归结构，且 $n + 1$ 与 n 仅相差一个常数阶，对整体结论的量化影响较小。

定理 4.16. 向量自内积的误差界

```
theorem fl_vec_dot_bound
  (n : ℕ)
  (hnu : ((n : ℝ) + 1) * (u : ℝ) < 1)
  (x : Vec n) (z : ℝ)
  (hfl : fl_vec_dot u x x z) :
    diff_gamma_bound u (n + 1) (x · x) z
```

本定理给出了向量与其自身内积的浮点计算结果的误差界。前提条件为：向量维度 n 满足 $(n + 1)u < 1$ （以保证 γ_{n+1} 有定义），且 z 是 x 与自身的带浮点误差的点乘结果。在此条件下，结论为

$$|z - x^T x| \leq \gamma_{n+1} |x^T x|.$$

证明. 记 $s_i = \sum_{k=1}^i x_k x_k$ 为精确的部分和（ $s_n = x^T x$ ）， $\hat{s}_i$ 为对应的浮点计算结果。由浮点运算定义展开递推关系：

$$\begin{aligned} \hat{s}_1 &= \text{fl}(0 + \text{fl}(x_1 x_1)) \\ &= \text{fl}(x_1 x_1)(1 + \theta_1) \\ &= x_1 x_1(1 + \theta_2), \\ \hat{s}_i &= \text{fl}(\widehat{s_{i-1}} + \text{fl}(x_i x_i)) \\ &= (\widehat{s_{i-1}} + x_i x_i(1 + \theta_1))(1 + \theta_1) \\ &= (s_{i-1}(1 + \theta_i) + x_i x_i(1 + \theta_1))(1 + \theta_1) \\ &= (s_{i-1} + x_i x_i)(1 + \theta_{i+1}) \\ &= s_i(1 + \theta_{i+1}). \end{aligned}$$

由此递推可得 $\hat{s}_n = s_n(1 + \theta_{n+1})$ ，且 $|\theta_{n+1}| \leq \gamma_{n+1}$ ，故 $|z - x^T x| \leq \gamma_{n+1} |x^T x|$ 。 $\square$

引理 4.17. 开方运算的误差传递

```
lemma sqrt_trans_gamma_bound (n : ℕ) (x x' : ℝ) (hx : x ≥ 0)
  (hnu : 2 * (n : ℝ) * (u : ℝ) < 1)
  (h_bound : diff_gamma_bound u n x x') :
    diff_gamma_bound u n (Real.sqrt x) (Real.sqrt x')
```

本引理考察开方运算对误差的影响。前提条件： x 为非负实数（保证开方有意义），其计算值 x' 满足 n 阶误差界（即 $x' = x(1 + \theta_n)$ ， $|\theta_n| \leq \gamma_n$ ），且 $2nu < 1$ （以保证 γ_n 有定义且误

差足够小)。在此条件下，开方结果 $\sqrt{x'}$ 相对于精确值 $\sqrt{x}$ 仍满足 n 阶误差界，即

$$|\sqrt{x'} - \sqrt{x}| \leq \gamma_n \sqrt{x}.$$

从理论上讲，开方运算具有压缩误差的作用：若 $x' = x(1 + \delta)$ ，则 $\sqrt{x'} \approx \sqrt{x}(1 + \delta/2)$ ，即误差阶数可减半至 $n/2$ 。然而，为保持整个误差分析框架中公式表达的一致性，并简化形式化证明的结构，我们在此处不进行这一缩紧，而是继续沿用 n 阶作为误差上界。这一保守处理不影响最终结论的定性正确性，只是会给出一个略为宽松的常数界。

引理 4.18. 取倒数运算的误差传递

```
lemma div_inv_trans_gamma_bound (n : N) (x x' : R) (hx : x > 0)
  (hnu : 2 * (n : R) * (u : R) < 1)
  (h_bound : diff_gamma_bound u n x x') :
  diff_gamma_bound u (2 * n) (1 / x) (1 / x')
```

本引理考察取倒数运算对误差的影响。前提条件： x 为正实数（保证倒数有定义），其计算值 x' 满足 n 阶误差界，且 $2nu < 1$ 。在此条件下，倒数 $1/x'$ 相对于精确值 $1/x$ 满足 $2n$ 阶误差界，即

$$\left| \frac{1}{x'} - \frac{1}{x} \right| \leq \gamma_{2n} \frac{1}{|x|}.$$

在经典的数值分析中，通常利用近似等式 $1/(1 + \theta_n) \approx 1 - \theta_n$ （在 $u \ll 1$ 时成立），认为取倒数不会放大误差阶数。然而，在严格的形式化证明中，不等式 $1/(1 - \gamma_n) \leq 1 + \gamma_n$ 并非恒成立——它仅在 γ_n 足够小时近似成立。为了保证结论在形式化系统中的绝对严谨性，我们不得不将倒数运算的误差界放宽至 $2n$ 阶，使下面的不等式组成立。

证明. 只需证明

$$\begin{cases} 1/(1 - \gamma_n) \leq 1 + \gamma_{2n}, \\ 1/(1 + \gamma_n) \geq 1 - \gamma_{2n}. \end{cases}$$

代入 γ 的定义 $\gamma_k = \frac{ku}{1-ku}$ ，上两式化为

$$\begin{cases} 1/(1 - \frac{nu}{1-nu}) \leq 1 + \frac{2nu}{1-2nu}, \\ 1/(1 + \frac{nu}{1-nu}) \geq 1 - \frac{2nu}{1-2nu}. \end{cases}$$

由前提条件 $2nu < 1$ 可知各分母均为正。两边通分后，上述不等式等价于

$$\begin{cases} (1 - \frac{nu}{1-nu}) \cdot (1 + \frac{2nu}{1-2nu}) \geq 1, & (1) \\ (1 + \frac{nu}{1-nu}) \cdot (1 - \frac{2nu}{1-2nu}) \leq 1. & (2) \end{cases}$$

以下分别证明这两个不等式。

对式 (1):

$$\begin{aligned}
\left(1 - \frac{nu}{1 - nu}\right) \cdot \left(1 + \frac{2nu}{1 - 2nu}\right) &= 1 + \gamma_{2n} - \gamma_n - \gamma_n \gamma_{2n} \\
&\geq 1 + \gamma_{2n} - 2\gamma_n \\
&= 1 + \frac{2nu}{1 - 2nu} - \frac{2nu}{1 - nu} \\
&\geq 1.
\end{aligned}$$

对式 (2):

$$\begin{aligned}
\left(1 + \frac{nu}{1 - nu}\right) \cdot \left(1 - \frac{2nu}{1 - 2nu}\right) &= 1 - \gamma_{2n} + \gamma_n - \gamma_n \gamma_{2n} \\
&= 1 + (\gamma_n - \gamma_{2n}) - \gamma_n \gamma_{2n} \\
&\leq 1.
\end{aligned}$$

□

上述结论对负实数 x 同样成立（只需取相反数后处理），但在本项目中，Householder 算法中涉及的取倒数操作仅作用于正实数 p （ $p = s \cdot v_0$ ，由引理可证其为正），因此我们未单独证明负数情形。

以上内容完整涵盖了源文件 Basic.lean 中关于浮点运算误差的基础定义与核心引理，包括 γ 函数的可加性、标量和向量的误差界谓词、基本运算的浮点误差封装、向量点乘的误差估计，以及开方和倒数运算的误差传递规律等结论。这些结论构成了整个 Householder 算法误差分析的理论工具箱。在下一节中，我们将借助这些工具，对 Householder 过程的每一步中间变量逐层进行误差追踪，最终给出输出向量的总体误差界。

4.3 Householder 过程

定义 4.19. Householder 变换的目标向量

```
def sigma_e1 (n : N) (x : Vec n) (hn : n ≠ 0) : Vec n :=
  fun i => if i = <0, Nat.pos_of_ne_zero hn>
    then - sign x hn * Real.sqrt (x · x) else 0
```

如第 2.2.3 小节所述，在构造 Householder 变换时，我们需要找到一个正交反射矩阵 P ，使得 $Px = \sigma e_1$ ，其中 $e_1 = (1, 0, \dots, 0)^T$ 是第一个分量上的单位向量，而 σ 是一个符号选择后的标量。我们选取的目标向量为

$$\sigma e_1 = -\text{sign}(x_1) \|x\|_2 e_1.$$

换言之，该向量仅第一个分量非零，其值为 $-\text{sign}(x_1) \|x\|_2$ ，其余分量均为 0。在 Lean 代码中，`Real.sqrt (x · x)` 即对应 $\|x\|_2$ （因为 $x^T x = \|x\|_2^2$ ），而 `sign x hn` 即第 4.1 节定义的符号函数。整个函数逐分量定义：当指标 i 表示第一个位置时（在 Lean 中是 $i = 0$ ），返回 $-\text{sign}(x) \|x\|_2$ ，否则返回 0。

定义 4.20. Householder 算法

对于任意非零向量 $x \in \mathbb{R}^n$, Householder 算法通过以下七步计算构造反射矩阵所需的向量 v , 使得 $P = I - vv^T$ 满足 $Px = -\text{sign}(x_1)\|x\|_2 e_1$:

1. 计算向量模长的平方: $l' = x^T x$ 。
2. 计算模长: $l = \sqrt{l'}$ 。
3. 引入符号: $s = \text{sign}(x) \cdot l$, 即 $s = -\sigma$ 。
4. 构造 v 的首分量: $v_1 = x_1 + s$ 。注意由 sign 的定义, s 与 x_1 符号相反, 因此 $v_1 \neq 0$ 。
5. 计算辅助标量: $p = s \cdot v_1$ 。
6. 计算 $\beta = 1/p$, 再计算 $b = \sqrt{\beta}$ 。
7. 定义向量 v' : $v'_1 = v_1$, $v'_i = x_i$ ($i > 1$), 再令 $v = b v'$ 。

将上述计算过程整理为方程组的形式如下:

$$\begin{cases} l' = x^T x, \\ l = \sqrt{l'}, \\ s = \text{sign}(x) \cdot l, \\ v_1 = x_1 + s, \\ p = s v_1, \\ \beta = 1/p, \\ b = \sqrt{\beta}, \\ v'_i = \begin{cases} v_1, & i = 1, \\ x_i, & i > 1, \end{cases} \\ v = b v'. \end{cases}$$

在 Lean 中, 我们使用了一个结构体 `Householder` 来封装这 9 个中间变量, 并定义了函数 `householder` 来实现上述计算过程。该函数逐一计算各中间变量, 并将结果打包为结构体返回。具体 Lean 代码如下:

```
structure Householder (n : ℕ) (hn : n ≠ 0) where
  l'  : ℝ
  l   : ℝ
  s   : ℝ
  v0  : ℝ
  v'  : Vec n
  p   : ℝ
  β   : ℝ
  b   : ℝ
  v   : Vec n
```

```

def householder (n : N) (x : Vec n) (hn : n ≠ 0) :
  Householder n hn := by
    let i₀ : Fin n := ⟨0, Nat.pos_of_ne_zero hn⟩
    let l' := x · x
    let l := Real.sqrt l'
    let s := sign x hn * l
    let v0 := x i₀ + s
    let p := s * v0
    let β := 1 / p
    let b := Real.sqrt β
    let v' : Vec n := fun i => if i = i₀ then v0 else x i
    let v := b · v'
    exact {l' := l', l := l, s := s, v0 := v0,
      v' := v', p := p, β := β, b := b, v := v
    }

```

定理 4.21. Householder 算法的正确性

```

theorem sigma_e₁_of_householder
  (n : N)
  (x : Vec n)
  (hn : n ≠ 0)
  (hx : x ≠ 0) :
  let hv := householder n x hn
  let P := 1 - (vecMulVec hv.v hv.v)
  P · x = sigma_e₁ n x hn

```

本定理是本节的核心结论，它断言上述 Householder 算法的正确性。前提条件： $n \neq 0$ （保证向量非空），且 x 为非零向量（若 $x = 0$ 则反射矩阵无意义）。在此条件下，令 hv 为算法返回的结构体，令 $P = I - vv^T$ 为所构造的反射矩阵（代码中 `vecMulVec hv.v hv.v` 即外积 vv^T ），则 P 作用于 x 的结果恰好等于目标向量 σe_1 。即

$$(I - vv^T)x = -\text{sign}(x_1)\|x\|_2 e_1.$$

证明. 我们将通过代数推导验证这一结论。首先，考察向量 v' 的表达式：由 v' 的定义及 v_1 和 s 的关系式，反复代入可得

$$v' = x - (-\text{sign}(x_1)\|x\|_2 e_1) = x - \sigma e_1.$$

因此 v' 恰为输入向量 x 与目标向量 σe_1 之差。由 Householder 矩阵的性质（第 2.1.2 节），

对于 v' 构造的标准反射矩阵 $I - \frac{2}{v'^T v'} v' v'^T$ ，我们有

$$\left(I - \frac{2}{v'^T v'} v' v'^T\right) x = \sigma e_1 = -\text{sign}(x_1) \|x\|_2 e_1.$$

接下来证明经过缩放后的 $v = bv'$ 构造的矩阵 $I - vv^T$ 与上述标准形式相同。由定义 $p = s \cdot v_1$ 以及 s 和 v_1 的表达式，计算可得

$$v'^T v' = x^T x + 2sx_1 + s^2 = 2sx_1 + 2s^2 = 2p.$$

从而

$$b = \sqrt{\beta} = \sqrt{\frac{1}{p}} = \sqrt{\frac{2}{v'^T v'}}.$$

令 $v = bv'$ ，则

$$v^T v = b^2 v'^T v' = \frac{2}{v'^T v'} \cdot v'^T v' = 2.$$

因此

$$I - vv^T = I - \frac{2}{v'^T v'} v' v'^T.$$

等式右侧正是以 v' 构造的标准 **Householder** 反射矩阵，而我们已经知道该矩阵能将 x 映射到 σe_1 ，故左侧的矩阵 $I - vv^T$ 同样具有这一性质。最后一式中的缩放因子 b 实质上起到归一化的作用：它将 v' 缩放到 $v^T v = 2$ ，使得反射矩阵的表达形式从 $I - \frac{2}{v'^T v'} v' v'^T$ 简化为 $I - vv^T$ ，这一简化形式在后续的误差分析中更为便于处理。□

上述推导在数学层面上完整地验证了 **Householder** 算法的正确性。在 **Lean** 形式化系统中，这一证明被拆解为多个更基本的命题和代数恒等式，通过策略组合逐级完成。完整的机器可验证证明代码见源文件 `Householder.lean`。

4.4 Householder 法的误差估计

定义 4.22. 带误差的 **Householder** 过程

```
def fl_householder (n : N) (x : Vec n) (hn : n ≠ 0)
  (hv_fl : Householder n hn) : Prop :=
  -- (1) l' = fl(xTx)
  fl_vec_dot u x x hv_fl.l' ^
  -- (2) l = fl(sqrt(l'))
  fl u (Real.sqrt hv_fl.l') hv_fl.l ^
  hv_fl.s = sign x hn * hv_fl.l ^
  -- (3) v_o = fl(x_o + s)
  fl_add u (x <0, Nat.pos_of_ne_zero hn>) hv_fl.s hv_fl.v0 ^
  -- (4) p = fl(s * v_o)
  fl_mul u hv_fl.s hv_fl.v0 hv_fl.p ^
  -- (5) β = fl(1 / p)
  fl u (1 / hv_fl.p) hv_fl.β ^
```

```

-- (6)  $b = fl(\sqrt{\beta})$ 
fl u (Real.sqrt hv_fl. $\beta$ ) hv_fl.b ^
-- (7)  $v = fl(b \cdot v')$ 
hv_fl.v' = (fun i => if i = <0, Nat.pos_of_ne_zero hn>
            then hv_fl.v0
            else (x i)) ^
fl_smul_vec u hv_fl.b hv_fl.v' hv_fl.v

```

上述代码以相继传递的中间变量形式，构建了考虑浮点误差扰动的 Householder 计算序列模型。在此链式结构中，每层衍生变量均须满足基于其直接前驱量的单次运算浮点误差。接下来的证明中，将逐层追踪误差在公式链中的放大特征，最终界定总体输出误差。

定理 4.23. Householder 算法误差的界 ([1] 引理 19.1)

```

theorem Lemma_19_1 (n : N) (x : Vec n)
  (hn : n ≠ 0) (hx : x ≠ 0)
  (hv_fl : Householder n hn)
  (hnu : (8 * n + 26 : R) * (u : R) < 1)
  (hfl : fl_householder u n x hn hv_fl) :
  diff_gamma_bound_vec u (5 * n + 18)
  (householder n x hn).v hv_fl.v

```

该引理证明了对于输出向量各分量均存在 θ_i 满足：在误差界 $|\theta_i| \leq \gamma_{5n+18}$ 条件下取得 $\hat{v}_i = v_i(1 + \theta_i)$ ($1 \leq i \leq n$)。

证明. 此处按计算顺序分步证明各步的具体累积误差界。作为参照基础，先行实例化不含浮点波动的精确过程变量结构 hv：

```
let hv := householder n x hn
```

基于 l' 的定义与定理 4.16，得到：

```
have l'_bound : diff_gamma_bound u (n + 1) hv.l' hv_fl.l'
```

由引理 4.17，开方运算不放大输入误差的上界。叠加本次浮点开方运算引入的一次误差，得到变量 l 的误差界：

```
have l_bound : diff_gamma_bound u (n + 2) hv.l hv_fl.l
```

由于符号函数取值仅为 1 或 -1，不引入额外浮点误差，因此 s 的误差界与 l 相同：

```
have s_bound : diff_gamma_bound u (n + 2) hv.s hv_fl.s
```

由 s 的定义可知， s 与 x_1 同号。依据引理 4.9，同号相加的误差界由操作数的最大误差界决定。加上本次浮点加法的误差，得到 v_0 的误差界：

```
have v0_bound : diff_gamma_bound u (n + 3) hv.v0 hv_fl.v0
```

由上文，变量 s 有 $n+2$ 阶界，变量 v_0 有 $n+3$ 阶界。依据引理 4.10，乘积 $s \cdot v_0$ 有 $2n+5$ 阶界。加上本次浮点乘法操作的误差，得到变量 p 的界：

```
have p_bound : diff_gamma_bound u (2 * n + 6) hv.p hv_fl.p
```

由引理 4.18，取倒数后误差阶数翻倍，得出倒数 $1/p$ 的误差界：

```
have p_bound' : diff_gamma_bound u (4 * n + 12)
    (1 / hv.p) (1 / hv_fl.p)
```

因 $\hat{\beta} = fl(\frac{1}{\beta})$ ，增加一次浮点赋值误差，得到 β 的界：

```
have beta_bound : diff_gamma_bound u (4 * n + 13) hv.beta hv_fl.beta
```

与计算 l 的步骤相似，开根运算不放大误差阶数。叠加本次开根号的浮点误差后，得到变量 b 的界：

```
have b_bound : diff_gamma_bound u (4 * n + 14) hv.b hv_fl.b
```

最后计算输出向量的分量误差。对于首位以外的分量 $i > 1$ ，由式 $\hat{v}_i = fl(\hat{b} \cdot x_i)$ 及乘法引理 4.10，由于 x_i 为精确常量， $\hat{v}_i$ 的误差界为 $4n + 15$ 阶。对于首位分量， $\hat{v}_1 = fl(\hat{b} \cdot \hat{v}_0)$ ，由于 $\hat{v}_0$ 带有 $n + 3$ 阶误差，故由乘法定理知 $\hat{v}_1$ 的误差界为 $5n + 18$ 阶。为统一定义，取各个分量的最大误差界，即最终输出向量的所有分量均被 $5n + 18$ 阶常数界控制。 $\square$

第五章 总结

5.1 工作总结

本文基于 Lean 4 交互式定理证明器，对数值计算中经典的 Householder 变换的数值稳定性进行了形式化验证。全文的主要工作与贡献可总结为以下三个方面：

首先，梳理了浮点数相对误差的标准模型，并在 Lean 4 中严格定义了一系列基础运算的浮点误差界（即 γ_n 误差界）。通过将误差控制过程抽象为形式化的引理和定理，验证了误差的可加性、保号性、传递性以及基本四则运算的误差传递规律，为复杂的数值算法分析奠定了理论基础。

其次，针对 Higham 在 *Accuracy and Stability of Numerical Algorithms* [1] 一书中给出的 Householder 算法误差分析过程（引理 19.1），进行了完整的形式化重构。在证明推导中，我们利用机器证明系统的严密性，发现并修补了部分在传统纸笔证明中依赖于“误差近似”或“常规假设”的逻辑间隙。例如，在取倒数运算中，将误差界适当地由 n 阶放宽至 $2n$ 阶，以满足形式化逻辑体系的全域成立条件；在向量自内积中，明确了首项附加加法所带来的阶数放宽。最终，我们严格证明了带误差的 Householder 过程所输出的向量分量统一受控于 $5n + 18$ 阶的综合误差界，确切地用机器形式化证明肯定了该经典算法的数值稳定性。

最后，本文的实践展示了交互式定理证明器在数值线性代数领域的应用潜力。形式化验证不仅能够发现和规避传统推导中的隐蔽预设，更为开发高可靠性、高精度的数值计算软件库提供了纯粹的理论证明背书。

5.2 未来展望

形式化数值分析是一个极具挑战但充满应用前景的领域。尽管本文已完成了 Householder 变换基本过程的误差分析，但受限于篇幅，仍有许多值得继续拓展的方向：

其一，本文主要聚焦于单次 Householder 变换的误差界限估计。在未来的研究中，可将此工作向后延伸，完成对完整 QR 分解算法乃至 SVD 等复合并行矩阵算法的形式化误差分析。

其二，目前定义的误差界为了保证系统证明的可组合性与顺畅性，在某些运算环节（如开方、倒数运算等）采用了相对保守的放缩策略。随着数值形式化理论库的不断完善，未来有望引入更精细的分析工具与阶数约束结构，从而推导出更加紧凑且贴近实际硬件舍入表现的误差上界。

其三，本文所构建的浮点运算模型和各类误差传递引理可进一步通用化与模块化，争取在未来为 Mathlib4 等主流开源数学库的数值分析分支补充基础定理，为广泛的机器形式化数值计算生态贡献代码与成果。

参考文献

- [1] Nicholas G. Higham. *Accuracy and Stability of Numerical Algorithms* (2nd ed.). Society for Industrial and Applied Mathematics, 2002.
- [2] Guoxiong Gao, Haocheng Ju, Jiedong Jiang, Zihan Qin, and Bin Dong. *A Semantic Search Engine for Mathlib4*. EMNLP 2024 Findings, 2024. <https://arxiv.org/abs/2403.13310>
- [3] Lean and Mathlib4 Maintainer team. Documentation page for Mathlib4 and Lean. https://leanprover-community.github.io/mathlib4_docs

附录

本文涉及的所有 Lean 4 形式化代码与 $\text{\LaTeX}$ 源码均已开源。使用的 Lean 4 工具链为 `leanprover/lean4:v4.30.0-rc2`, 项目代码库托管于 GitHub, 地址为: <https://github.com/kfc2333/Householder>。

项目的主要文件内容与目录结构如下:

- QR/目录: 存放所有的 Lean 证明源码, 包括:
 - 基础误差模型定义 (`Basic.lean`)
 - 精确 Householder 过程 (`Householder.lean`)
 - 带误差界限的完整推导 (`Householder_fl.lean`)
- docs/目录: 存放本文档的排版源文件。

在本地编译运行本项目, 建议首先安装 Lean 官方的环境管理工具 `elan`, 系统会自动读取工具链文件获取指定的 Lean 4 编译器。随后在项目根目录依次执行以下命令:

```
lake exe cache get
lake build
```

其中 `cache get` 命令用于拉取数学库依赖的预编译缓存。若 `lake build` 编译过程无错误输出, 即代表全部证明均已通过 Lean 4 的形式化机器验证。

致谢

北京大学学位论文原创性声明和使用授权说明

原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品或成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本声明的法律结果由本人承担。

论文作者签名：

日期： 年 月 日

学位论文使用授权说明

本人完全了解北京大学关于收集、保存、使用学位论文的规定，即：

- 按照学校要求提交学位论文的印刷本和电子版本；
- 学校有权保存学位论文的印刷本和电子版，并提供目录检索与阅览服务，在校园网上提供服务；
- 学校可以采用影印、缩印、数字化或其它复制手段保存论文；

论文作者签名：

导师签名：

日期： 年 月 日